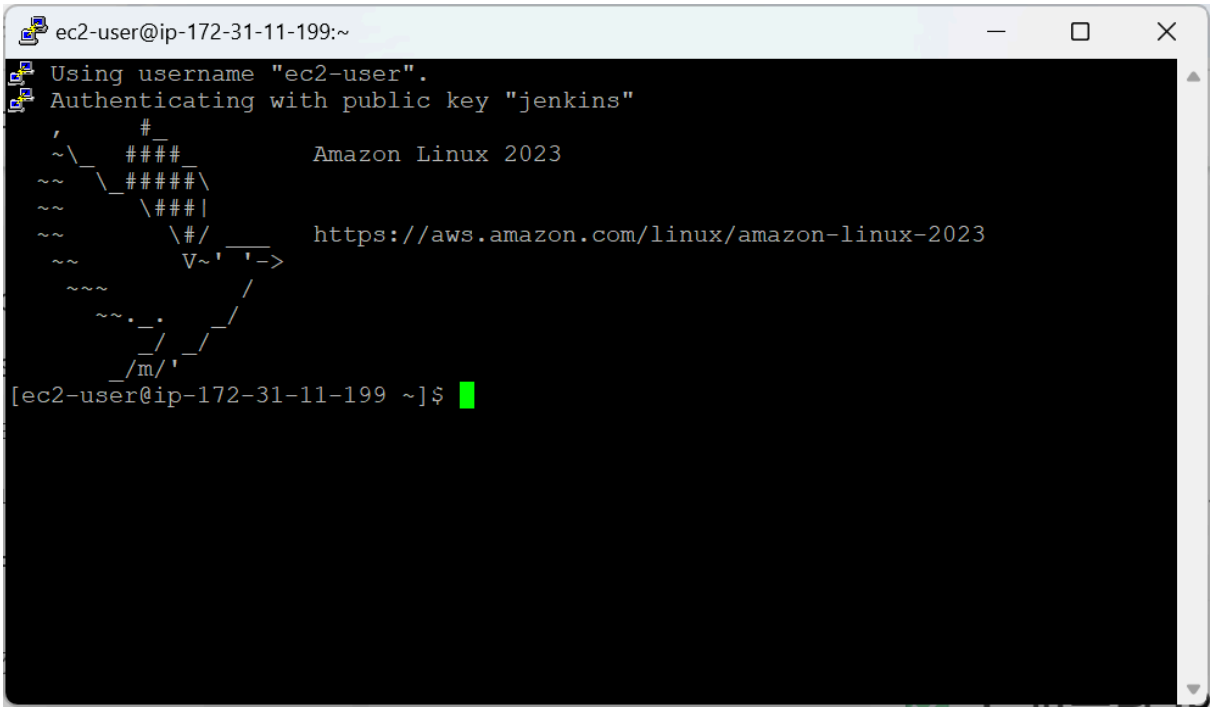
4. Jenkins를 활용한 CI/CD 환경 구성 - Jenkins 구성 및 웹 앱 배포

Git과 Jenkins 서버를 연동한 로컬 배포 실습입니다.

1. EC2 서버(Jenkins Server)에 Jenkins 설치

- PuTTY를 통한 EC2 접속



- Java 17 설치 및 경로 설정(Jenkins는 자바 환경에서 실행됨)

```
# 자바 17 Amazon Corretto 설치
$ sudo yum install -y java-17-amazon-corretto-devel

# 자바 설치 확인
$ java -version
# → openjdk 17 버전이 정상적으로 설치되었는지 확인

# Apache Maven 3.9.12 다운로드
```

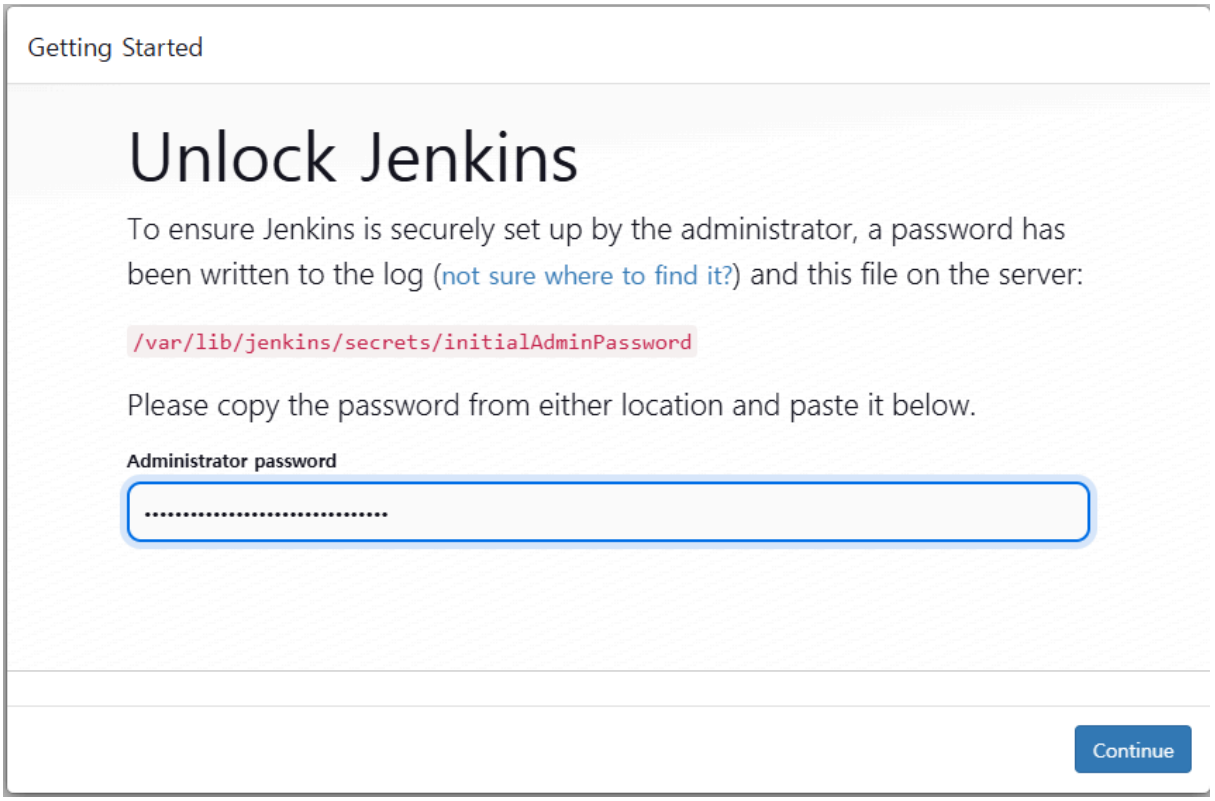
```
$ wget https://dlcdn.apache.org/maven/maven-3/3.9.12/binari
es/apache-maven-3.9.12-bin.tar.gz

# 압축 해제
$ tar -xvzf apache-maven-3.9.12-bin.tar.gz

# Maven 디렉터리를 /opt 아래로 이동
$ sudo mv apache-maven-3.9.12 /opt/maven

# JAVA_HOME 환경 변수 설정
$ echo "export JAVA_HOME=/usr/lib/jvm/java-17-amazon-corret
to.x86_64" >> ~/.bashrc
$ echo "export PATH=\$JAVA_HOME/bin:\$PATH" >> ~/.bashrc
$ source ~/.bashrc  # 변경된 환경 변수 적용

# MAVEN_HOME 환경 변수 설정
$ echo "export MAVEN_HOME=/opt/maven" >> ~/.bashrc
$ echo "export PATH=\$MAVEN_HOME/bin:\$PATH" >> ~/.bashrc
$ source ~/.bashrc  # 변경된 환경 변수 적용

# 환경 변수 확인
$ echo $JAVA_HOME    # → Java 설치 경로 확인
$ echo $MAVEN_HOME   # → Maven 설치 경로 확인
```

- Jenkins 및 Git 설치

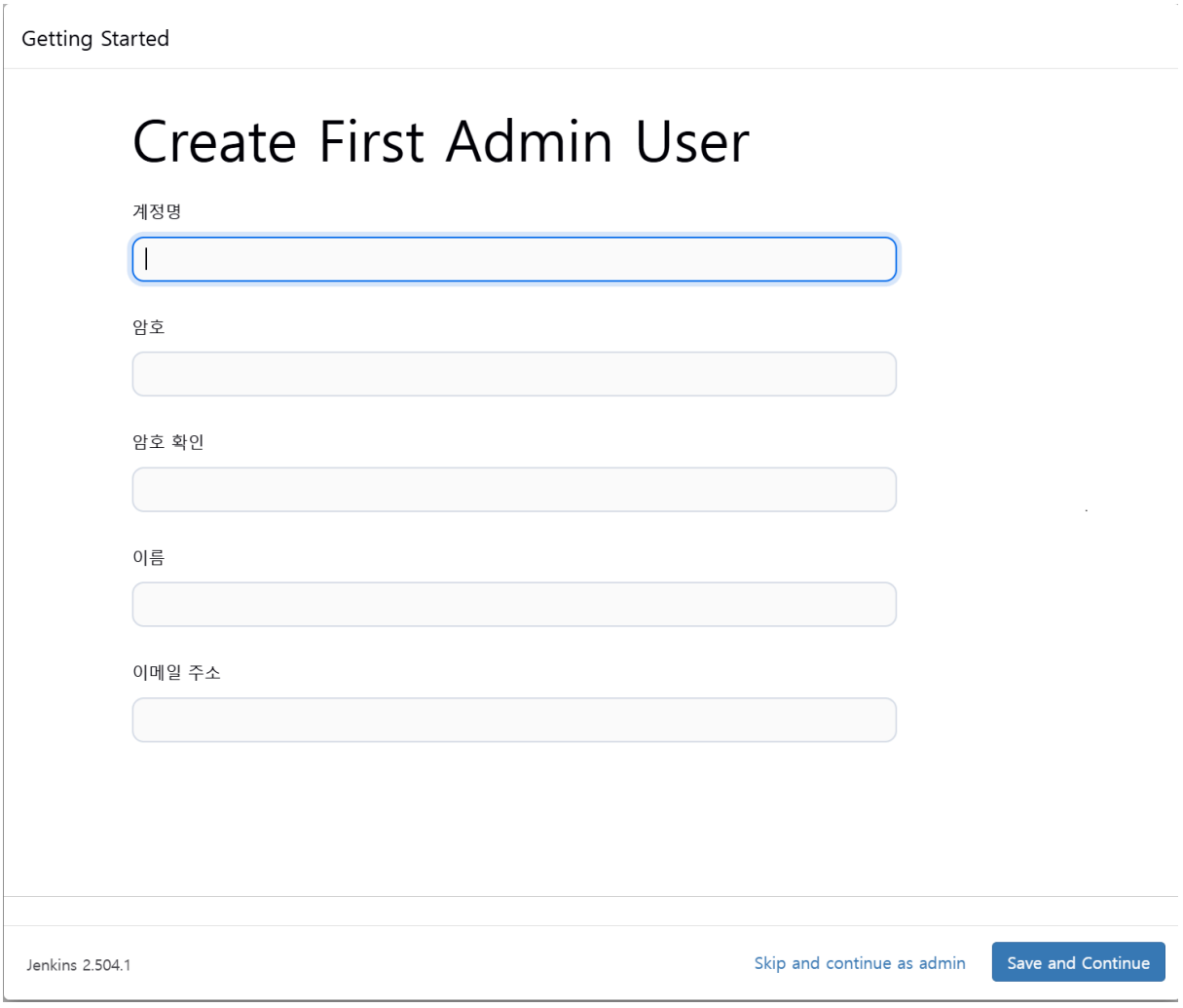
```
# Jenkins 저장소 등록
$ sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.je
nkins.io/redhat-stable/jenkins.repo

# Jenkins 공개키 등록
$ sudo rpm --import https://pkg.jenkins.io/redhat-stable/je
nkins.io-2023.key

# Jenkins 설치
$ sudo yum install jenkins -y
```

```
# systemd 데몬 재로드 (서비스 정보 갱신)
$ sudo systemctl daemon-reload

# Jenkins 서비스 부팅 시 자동 실행 설정
$ sudo systemctl enable jenkins

# Jenkins 서비스 시작
$ sudo systemctl start jenkins

# Jenkins 서비스 상태 확인
$ sudo systemctl status jenkins

# Ctrl + c 로 상태 종료

# Git 설치
$ sudo yum install git -y

# Jenkins 초기 관리자 비밀번호 확인 (웹 UI 최초 로그인 시 필요)
$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword

485b492b5f3d456c8aa77499e8476d53

# 마우스 드래그로 비밀번호 복사

# Jenkins 접속: 웹 브라우저에서 아래 주소로 접근
# Jenkins URL: http://{탄력적 아이피}:8080/
# → 위에서 복사한 초기 비밀번호 입력 → 관리자 계정 생성
```

- Jenkins 페이지 접속 및 설치 진행
- Jenkins URL 접속
- 초기 비밀번호 입력하여 설치 진행



- install suggested plugins 선택
- 계정정보입력



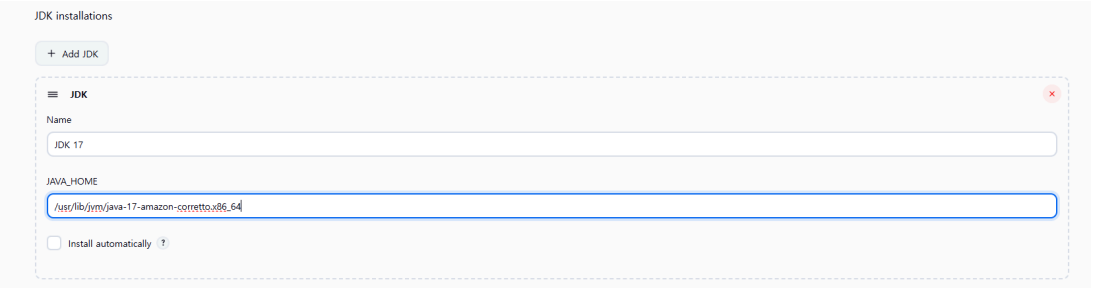
- 계정명 : cicd
- 비밀번호 : 1234

2. Jenkins 환경 설정

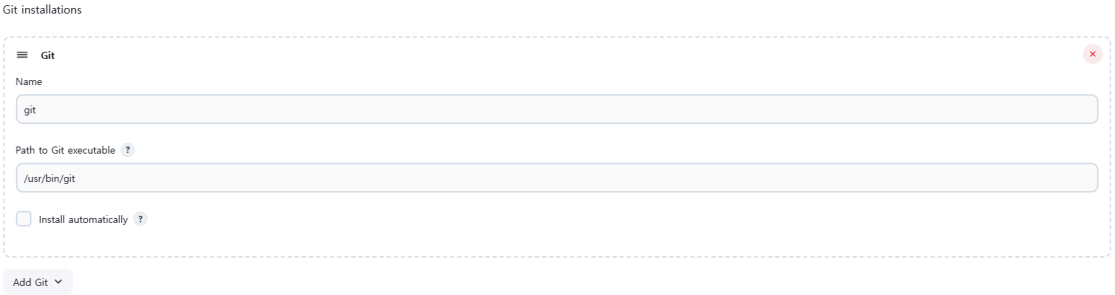
- 설정 > Jenkins 관리 > Tools
  - Maven 설정 - Use default maven settings

  - JDK installations - /usr/lib/jvm/java-17-amazon-corretto.x86_64
    - Name - JDK 17

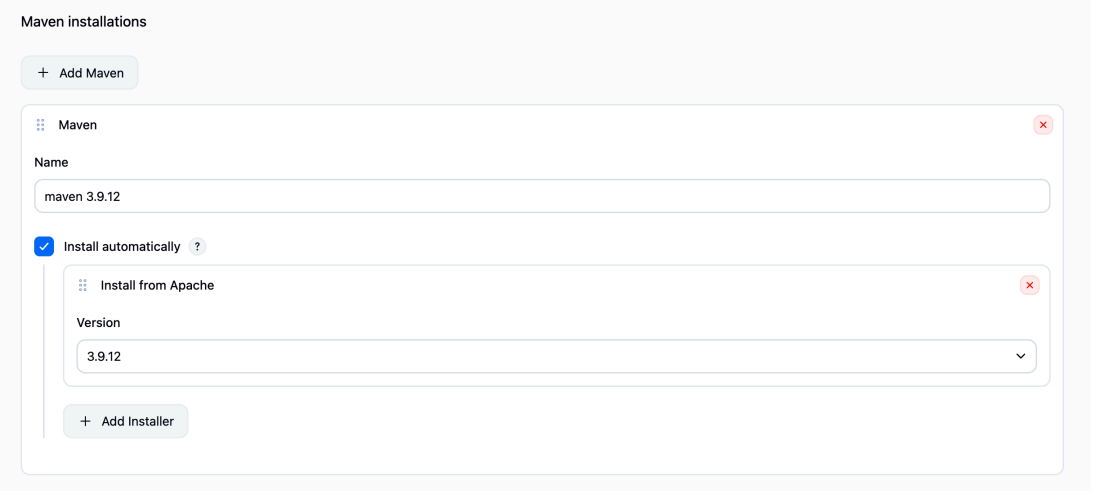
    - JAVA_HOME



- Git installations - /usr/bin/git
    - name - git
    - Path to Git executable - /usr/bin/git



- Maven installations(설치된 maven 버전과 일치하게)
- Maven Name은 이후에 jenkins pipeline에서 활용
    - name - maven 3.9.12



3. Git 연결

- **Jenkins가 설치된 EC2 서버에서 SSH 키 추가**

```
# git과 연결할 SSH 키 생성
$ sudo mkdir /var/lib/jenkins/.ssh
$ sudo ssh-keygen -t rsa -f /var/lib/jenkins/.ssh/jenkin
s_rsa_key
Enter passphrase : enter 입력
Enter same passphrase again : enter 입력
```

  - RSA란?
    - RSA는 현대 암호 시스템에서 가장 널리 사용되는 **공개 키 암호화(Public Key Cryptography) 알고리즘** 중 하나입니다.
    - RSA 암호화는 두 개의 서로 다른 큰 소수를 사용하며, 이 숫자들이 가진 수학적 특성을 기반으로 합니다.
    - 공개 키 암호화 (Public Key Cryptography) : RSA의 핵심 특징입니다. 전통적인 암호화 방식(대칭 키 암호화)과 달리, RSA는 정보를 암호화하고 복호화하는 데 **서로 다른 두 개의 키**를 사용합니다.
    - **공개 키 (Public Key):** 누구나 알 수 있도록 외부에 공개되는 키입니다. 데이터를 **암호화**하는 데 사용됩니다.
    - **개인 키 (Private Key):** 사용자 본인만 비밀리에 소유하는 키입니다. 암호화된 데이터를 **복호화**하는 데 사용됩니다.

  - jenkins_rsa_key(private key) : 내부 서버에 등록될 key

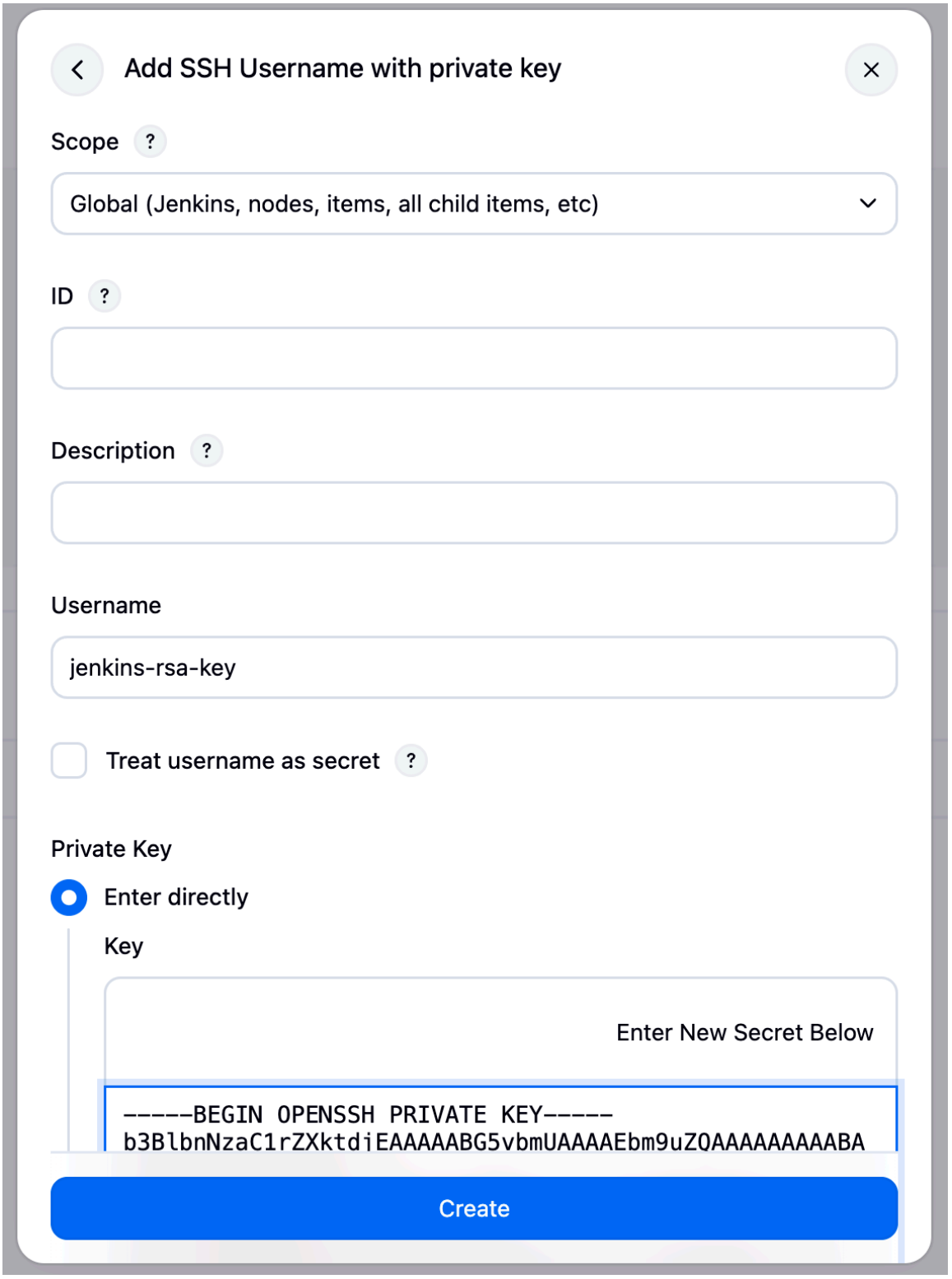
- jenkins_rsa_key.pub(public key) : 외부 서버에 등록될 ke

- key 값을 복사하여 jenkins에 등록

```
$ cd /var/lib/jenkins/.ssh
$ sudo cat jenkins_rsa_key
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAB
lwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEA0rpHVcn+i9NqhzLla+PZxdSDnferq/AKl4RFxP
+x12JNDhJO6Rzp
n1k1ydwZi1EWkClfrlxVTUAS/Nq2qdiIWUk5uIwipbW8HVrAu5Cnwn5f
YynJvxWAqEtJvy
                                -- 생략 --
-----END OPENSSH PRIVATE KEY-----
```

- 출력되는 rsa key 값을 복사
- 설정 > Jenkins 관리 > Credentials > Add Credentials 클릭 > SSH Username with private key 선택

- 아래와 같이 Credentials 정보를 입력(rsa key 값을 붙여넣기)

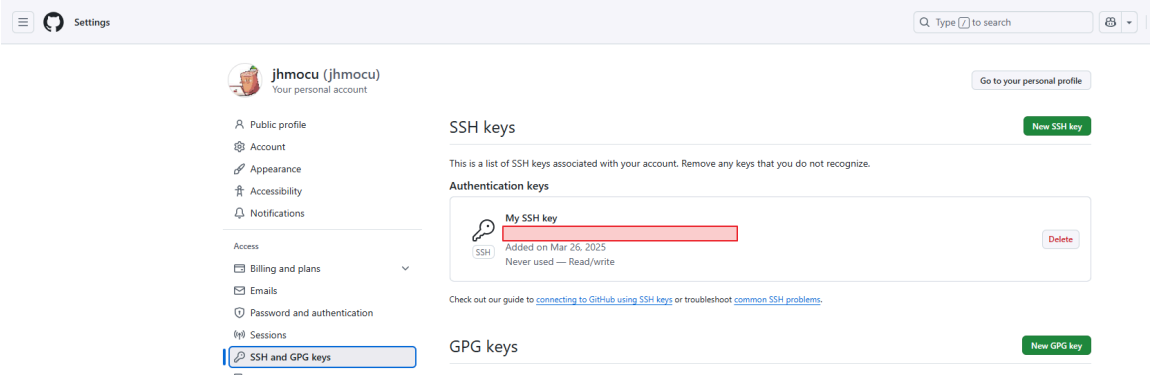


- Github SSH private Key 추가

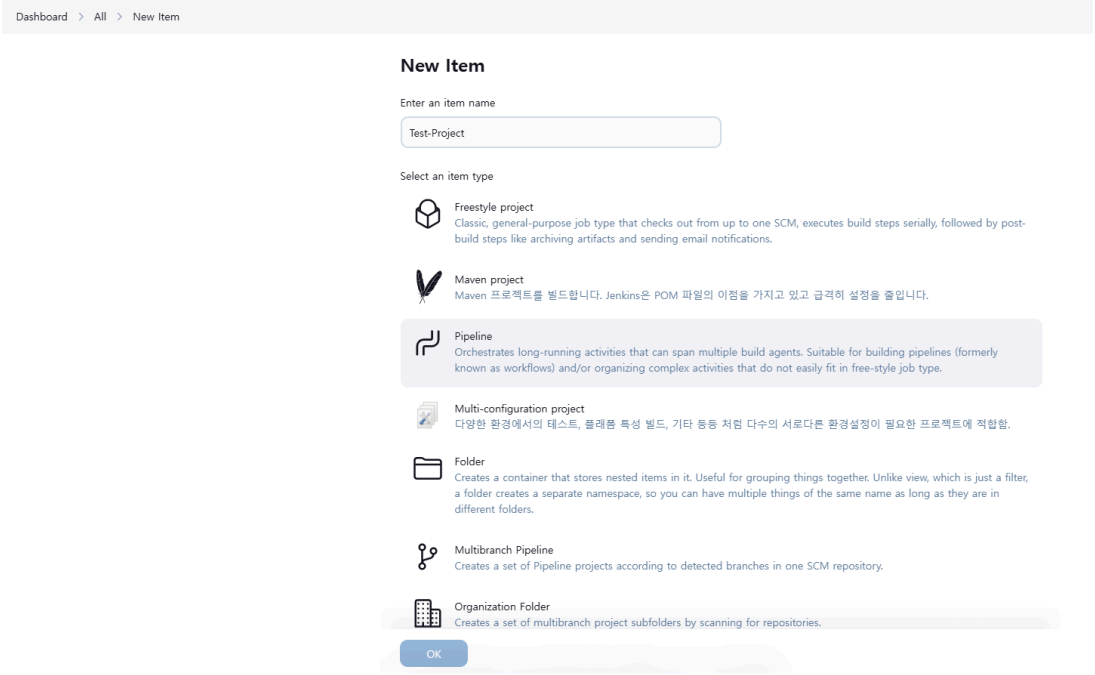
```
# rsa key가 생성되어 있는 .ssh 위치
$ cd /var/lib/jenkins/.ssh
```

```
$ sudo cat jenkins_rsa_key.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDSukdVyf6L02qHM
uVr49nF1IOd96ur8AqXhEXE/7HXYk0OEk7pHOmfWTXJ3BmLURaQKV
+uXFVNQBL82rap2IhZSTm4jCKltbwdWsC7kKfCfl9j
                                -- 생략 --
# ssh-rsa로 시작하는 값 복사
```

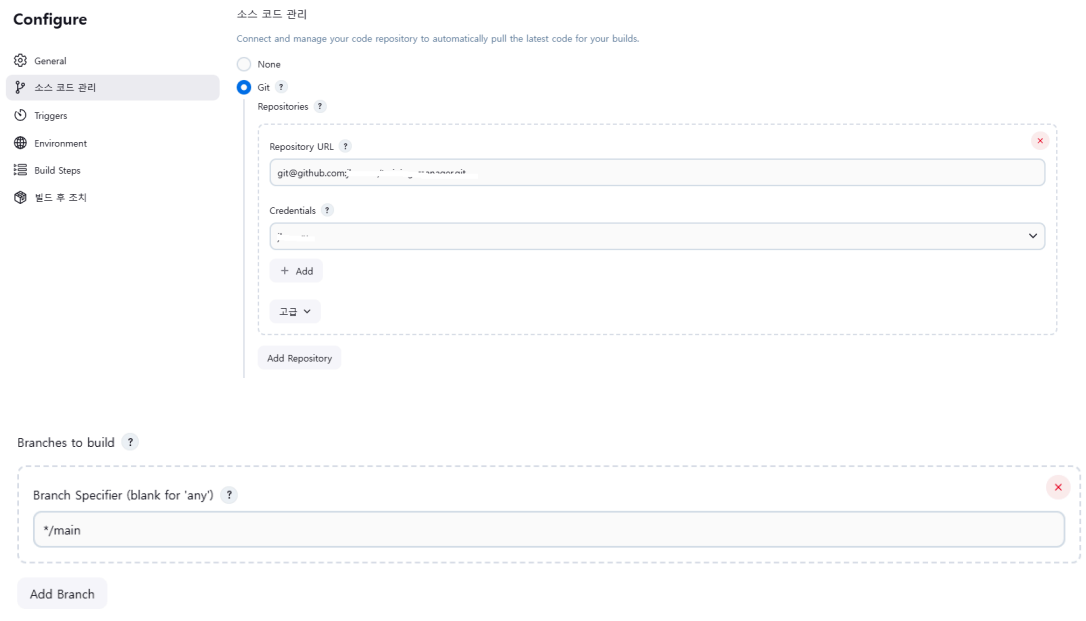
- Github → Profiles → Settings → SSH and GPG keys → New SSH Key → 복사한 key 값 붙여넣기

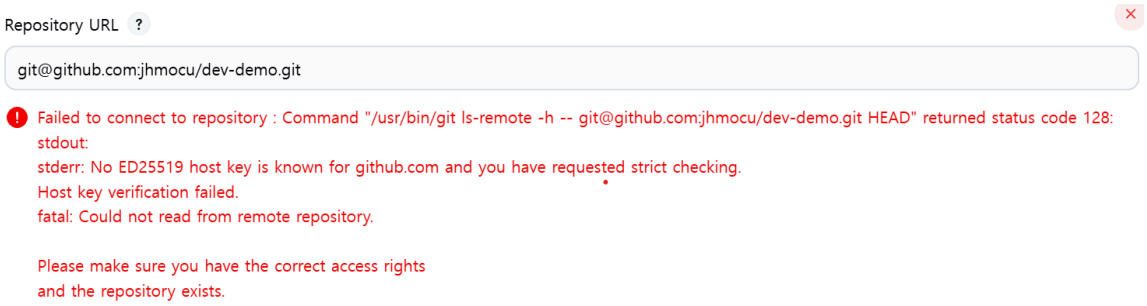


- Jenkins 페이지 이동 - 작업(item) 추가
  - Dashboard > 새로운 Item > name 작성 및 Freestyle project 선택



- Jenkins git repository 연결
  - Test-Project(item) → Configuration → 소스 코드 관리
  - Git 선택하고 URL 정보 입력 - git@github.com:jhmocu/mclass.git
  - Credentials 선택
  - Branches to build → */main





- 위와 같이 에러 발생 시 jenkins 서버에 아래와 같이 등록

```
# ssh github.com을 known_host로 등록
$ ssh-keyscan -t rsa github.com >> ~/.ssh/known_hosts

# 확인
```

```
$ cat ~/.ssh/known_hosts | grep github.com
github.com ssh-rsa AAAAB3NzaC1yc~~~
```

- 확인 후 출력된 github.com ssh ~~ 문자열을 복사하여 jenkins에 등록
- 설정 > Jenkins 관리 > Security 선택
- Git Host Key Verification Configuration - Manually Provided key에 위에서 출력된 github.com ssh-rsa 값 등록
- save
- jenkins 재부팅 및 확인

```
# Jenkins 서비스 재시작 명령어
$ sudo systemctl restart jenkins
```

- Test-Project(item) → Configuration → 소스 코드 관리
  Git 선택하고 URL 정보 입력 - git@github.com:jhmocu/mclass.git
  Credentials 선택

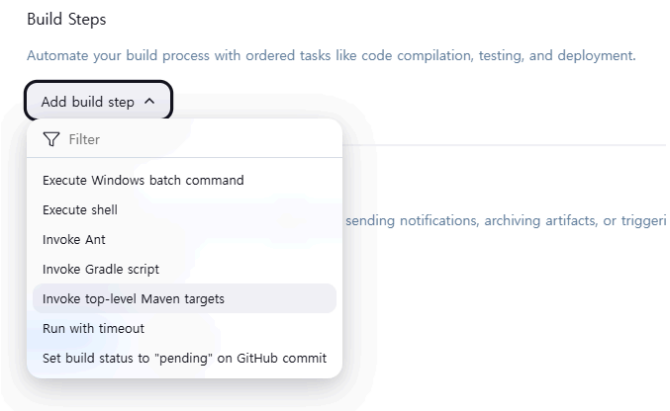
- EC2 프리티어 환경에서 Jenkins Node의 Swap Space와 Temp Space 용량 부족 문제
  - t2.micro에서 제공하는 RAM 용량 문제로 인해 빌드시 인스턴스가 실행되지 않음
  - Swap 공간을 늘려 추가적인 용량을 제공

```
# /tmp 디렉토리에 tmpfs 파일 시스템을 크기 2GB로 설정하여 마운트하도록 fstab 파일에 추가
# fstab 파일에 설정된 내용을 기반으로 /tmp 디렉토리의 tmpfs 파일 시스템을 다시 마운트
$ echo 'tmpfs /tmp tmpfs defaults,size=2G 0 0' | sudo tee -a /etc/fstab && sudo mount -o remount /tmp && df -h /tmp

# 루트 파일에 swapfile 생성 후 권한 부여
$ sudo dd if=/dev/zero of=/swapfile bs=128M count=16
# 기다려야 함

# 스왑 파일(swap file)의 권한을 변경
$ sudo chmod 600 /swapfile
```

```
# swap 영역 설정
$ sudo mkswap /swapfile

# swap 영역에 swapfile 추가 사용
$ sudo swapon /swapfile

# 확인
$ sudo swapon -s

# 부팅 시, 스왑 파일을 시작할 수 있도록 vim 에디터로 수정
$ sudo vi /etc/fstab

# vim 에디터 INSERT는 i 입력
# INSERT 나올 때는 esc 입력
# 저장 후 종료는 :wq! 입력

# 마지막 줄에 아래 입력하고 저장 후 종료
/swapfile swap swap defaults 0 0

# 추가 공간 확인
$ sudo free
# 재시작 필수
$ sudo reboot
```

5. 프로젝트 배포

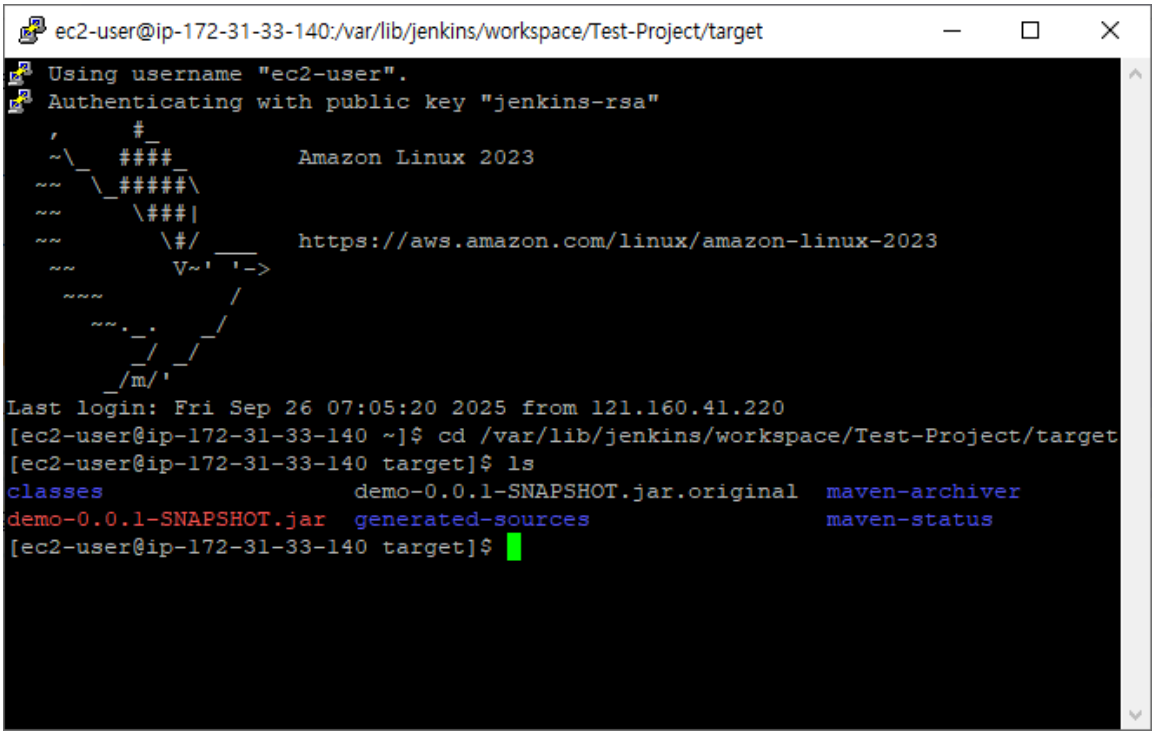
- Test-Project-Configuration-Build Steps 추가
  - 빌드 단계에서 필요한 기능을 추가하고, Maven을 활용하여 배포 파일을 생성하는 코드를 적용
  - Build Steps에 파라미터를 application.properties에 적용시키는 코드 추가



- Build Steps에 Maven 명령어 추가
  - Maven Version : maven 3.9.12
  - Goals : clean package -D maven.test.skip=true
    - 프로젝트를 완전히 초기화(Clean)하고, 단위 테스트를 실행하지 않은 채(Skip) 배포 가능한 패키지 파일(Package)을 생성하라는 의미

- Test-Project → 지금 빌드 수행
- 빌드를 수행하면 아래 경로에 jar 파일 생성됨

```
$ cd /var/lib/jenkins/workspace/Test-Project/target
$ ls
```



- jar 파일을 실행하여 서버 실행

```
$ sudo nohup java -jar /var/lib/jenkins/workspace/Test-Project/target/demo-0.0.1-SNAPSHOT.jar > /home/ec2-user/app.log 2>&1 &

# 이 명령어는 Jenkins에서 빌드된 Java 애플리케이션(JAR 파일)을 서버의 백그라운드에서 실행하며, 터미널 세션이 종료되어도 계속 실행되도록(데몬화) 로그를 특정 파일에 저장하는 표준적인 배포 명령입니다.

# 로그 실시간 확인
tail -f /home/ec2-user/app.log

# 로그 마지막 100줄만 확인
tail -n 100 /home/ec2-user/app.log

# 로그 전체 확인
cat /home/ec2-user/app.log
```

- AWS 인바운드 규칙 8081 포트 추가하고 접속

- 서버 접속이 완료되면 성공

4-1. EC2 프리티어 환경에서 Jenkins Node의 Swap 공간 부족 및 Temp Space 문제